เอกสารส่งมอบงาน (handover document)

ชื่อโครงการ / ระบบ:	โครงการพัฒนาระบบนำทางโดรนแบบเรียลไทม์ (giv-rt)
ผู้ส่งมอบงาน:	นายกรรชัย บัวงาม (รหัสนักศึกษา 6604101301)

ส่วนที่ 1: ภาพรวมงานและวัตถุประสงค์ (overview & context)

1.1 วัตถุประสงค์และเป้าหมายหลัก

- พัฒนาโปรแกรมรับข้อมูลจากเซ็นเซอร์ 3 ชนิด ได้แก่ กล้อง (camera), imu (icm-42688-p) และ gps (u-blox m10) เพื่อส่งต่อให้ระบบนำทางโดรนใช้งานแบบเรียลไทม์
- กำหนดให้โปรเซสอ่านค่าเซ็นเซอร์ทั้ง 3 ตัวทำงานร่วมกันบน cpu core 3 เพียงแกนเดียวทั้งหมด เพื่อลดเวลาแฝง (latency) และป้องกันปัญหา context switching จากระบบปฏิบัติการ
- ส่งออกข้อมูลผ่าน posix shared memory (/dev/shm) โดยแยกจุดส่งข้อมูลเป็น /video สำหรับข้อมูลกล้อง และ /sensor สำหรับข้อมูล imu/gps
  เพื่อให้โปรเซสอื่นนำไปใช้งานต่อได้ทันทีโดยไม่ต้องคัดลอกหน่วยความจำหลายขั้นตอน (zero-copy)
- ประเมินความเสถียรของเวลาการรับส่งข้อมูล (jitter) โดยนำค่าเวลาจริงจากระบบ (system timestamp) ในแต่ละรอบมาหักลบกัน (delta ts)

1.2 สถาปัตยกรรมระบบและลำดับการทำงาน

• ขั้นตอนการรับข้อมูล: ดึงข้อมูลจากกล้อง (csi), imu (spi) และ gps (uart) พร้อมกันบน cpu core 3
• ขั้นตอนบันทึกเวลา: บันทึกเวลา monotonic จริงจากระบบทันทีที่อ่านค่าเซ็นเซอร์ได้สำเร็จ
• ขั้นตอนการจัดส่งข้อมูล: เขียนข้อมูลลง shared memory แยกตาม endpoint (/video และ /sensor)
• ขั้นตอนประเมินผล: คำนวณความคลาดเคลื่อนของเวลา (jitter = |delta ts_actual - delta ts_target|) จาก timestamp จริงของระบบเพื่อตรวจสอบความเสถียร

ส่วนที่ 2: สถานะโครงการและรายการงาน (task & project status)

2.1 งานที่ดำเนินการเสร็จสิ้นแล้ว (completed tasks)

• พัฒนาโมดูลอ่านค่าเซ็นเซอร์ imu (icm-42688-p) ผ่าน spi (/dev/spidev0.0) และ gps (u-blox m10) ผ่าน uart สำเร็จ

• พัฒนาโมดูลอ่านและจับภาพกล้องเข้ารหัส png และส่งเข้า shared memory สำเร็จ

• ตั้งค่าการตรึงการทำงานของโปรเซสทั้งหมดให้อยู่บน cpu core 3 และกำหนดสิทธิ์ real-time (sched_fifo) สำเร็จ

• สร้าง shared memory endpoints (/video และ /sensor) และระบบคำนวณ jitter จาก system timestamp สำเร็จ

2.2 ตารางติดตามสถานะงาน (task status)

รายการงาน / โครงการ	สถานะ	สิ่งที่ต้องทำต่อ	กำหนดส่ง
ระบบอ่านและส่งข้อมูล imu/gps (c++)	เสร็จสิ้น (100%)	รวมการทดสอบระยะยาวร่วมกับ ros 2	เสร็จสิ้น
ระบบรับและส่งข้อมูลกล้อง (go)	เสร็จสิ้น (100%)	ปรับแต่งรอบเวลาประมวลผลบน core 3	เสร็จสิ้น
การทดสอบ jitter	อยู่ระหว่างทดสอบ (80%)	เก็บข้อมูลผลต่างเวลาเพื่อสร้างรายงานวิเคราะห์	กำลังดำเนินการ

2.3 งานค้างและแผนในอนาคต (backlog & future roadmap)

• นำไปทดสอบบน raspberry pi compute module 5 (cm5)

• ทดสอบการทำงานของระบบภายใต้สภาวะที่มีภาระงานสูง (load test) เพื่อดูความเสถียรของค่า jitter

ส่วนที่ 3: รายละเอียดเชิงเทคนิคและคู่มือการปฏิบัติงาน (technical details)

3.1 สถาปัตยกรรมซอฟต์แวร์และโครงสร้างไฟล์

• configsensor.toml: ไฟล์ตั้งค่าหลักของระบบ (ตั้งค่า cpu core, ความเร็ว serial, และพาทไฟล์ log)

• writer.cpp: โปรแกรมภาษา c++ สำหรับอ่านค่า imu ผ่าน spi/i2c เขียนลง /dev/shm/sensor (ตรึง cpu core 3)

• camcap.go: โปรแกรมภาษา go สำหรับอ่านค่ากล้อง เข้ารหัสภาพ png เขียนลง /dev/shm/video (ตรึง cpu core 3)

• receiver.cpp: โปรแกรมภาษา c++ สำหรับอ่านค่า gps ผ่าน uart เขียนลง /dev/shm/sensor (ตรึง cpu core 3)

3.2 เครื่องมือ เทคโนโลยี และการตั้งค่า real-time

• เครื่องมือหลัก: c++17, go 1.22+, ไลบรารี toml++, libdeflate (บีบอัด png), และ system calls ของ linux

• การกำหนด single-core: บังคับให้ทุกโปรเซสทำงานบน cpu core 3 และตั้งค่าลำดับความสำคัญ sched_fifo (priority 90)

• การจัดการหน่วยความจำ: ใช้ mlockall ป้องกันการ swap memory และปิดระบบประหยัดพลังงานเพื่อลด latency

• โครงสร้าง shared memory: แบ่งเป็น /dev/shm/sensor (เก็บ imudata และ gpsdata) และ /dev/shm/video (เก็บ png buffer)

3.3 ขั้นตอนการเปิดใช้งานและควบคุมระบบผ่าน makefile (system execution & lifecycle commands) ระบบรองรับการบริหารจัดการผ่านคำสั่ง **make** เพื่อช่วยอำนวยความสะดวกในการคอมไพล์ รันโปรเซสทั้งหมดบน **cpu core** เดียวกัน (**core 3)** และการเคลียร์ทรัพยากรระบบอย่างเป็นอัตโนมัติ ดังนี้ **1.** คำสั่งรันระบบแบบอัตโนมัติ (**sudo make run)**

ใช้สำหรับสั่งคอมไพล์ซอร์สโค้ดทั้งหมด (**c++** และ **go)** และเปิดใช้งานโปรเซสอ่านค่าเซ็นเซอร์ทุกตัวขึ้นมาทำงานร่วมกันบน **cpu core 3** โดยมีขั้นตอนการทำงานภายในดังนี้**:**

- เคลียร์ **shared memory** เดิม: ลบไฟล์หน่วยความจำความเร็วสูง **/dev/shm/video** และ **/dev/shm/sensor** ที่อาจค้างอยู่ออกก่อน
- รันโปรเซส **imu (writer):** สั่งรันโปรเซสอ่านค่า **imu** บน **cpu core 3 (taskset -c 3)** และส่งเข้าทำงานแบบ **background process**
- รันโปรเซส **gps (receiver):** หน่วงเวลา **0.5** วินาที แล้วสั่งรันโปรเซสอ่านค่า **gps** บน **cpu core 3 (taskset -c 3)** แบบ **background process**
- รันโปรเซสกล้อง (**camcap):** หน่วงเวลา **0.5** วินาที แล้วสั่งรันโปรเซสอ่านและจับภาพกล้องบน **cpu core 3 (taskset -c 3)** พร้อมโหลดไฟล์ตั้งค่า **cam/config.toml** และกำหนดค่า **godebug=asyncpreemptoff=1** เพื่อป้องกัน **go runtime** ขัดจังหวะการทำงานแบบ **real-time**
- **auto-cleanup on exit:** เมื่อโปรเซสกล้องหยุดทำงาน สคริปต์จะทำการส่งสัญญาณหยุดโปรเซส **writer** และ **receiver** ที่เหลืออยู่โดยอัตโนมัติทันที

2. คำสั่งหยุดการทำงานของระบบแบบบังคับ (**sudo make stop**)ใช้สำหรับหยุดโปรเซสการทำงานทั้งหมดเมื่อเกิดเหตุขัดข้องหรือต้องการปิดระบบอย่างเร่งด่วน โดยทำหน้าที่ดังนี้:

- ส่งสัญญาณ **kill -9** เพื่อบังคับปิดโปรเซสที่เกี่ยวข้องทั้งหมด ได้แก่ **writer, receiver, camcap** รวมถึงโปรเซสดึงภาพพื้นหลังอย่าง **rpicam-vid**
- ตรวจสอบ **process id (pid)** ที่ค้างอยู่ในระบบ

3. คำสั่งล้างไฟล์ไบนารีและแคชระบบ (**sudo make clean**)
ใช้สำหรับลบไฟล์ที่ไม่จำเป็นหลังจากจบการทำงานหรือก่อนเริ่มสั่งคอมไพล์ระบบใหม่ โดยทำหน้าที่ดังนี้:

- ลบไฟล์ไบนารีจากการคอมไพล์ (**writer, receiver, camcap**)
- ลบไฟล์บันทึกข้อผิดพลาด (***.log**), ไฟล์ **process id (*.pid)** และหน่วยความจำ **shared memory (/dev/shm/video, /dev/shm/sensor)**
- ลบโฟลเดอร์แคชของ **python (__pycache__, .cache)** ที่เกิดจากสคริปต์วิเคราะห์ผล

ส่วนที่ 4: ปัญหา ข้อควรระวัง และแนวทางแก้ไข (risks & troubleshooting)

4.1 ปัญหาที่อาจพบและข้อควรระวัง (**known issues & warnings**)

• การแย่งชิงทรัพยากรและการจัดลำดับความสำคัญ (**priority contention**): เนื่องจากโปรเซสทั้ง **3** ตัว (**writer, receiver, camcap**) ถูกตรึงให้ทำงานบน **cpu core 3** เดียวกันและใช้ลำดับความสำคัญ **real-time** สื่อสารระดับสูง (**sched_fifo priority 90**)หากโปรเซสกล้อง (**camcap**) ใช้ระยะเวลาประมวลผลหรือบีบอัดภาพยาวนานเกินไปอาจส่งผลกระทบให้โปรเซสอ่านค่าเซ็นเซอร์ที่มีความถี่สูงอย่าง **imu** เกิดการรอคอย (**latency drop**)หรือเกิดค่า **jitter** แกว่งผิดปกติในรอบนั้นๆ

• สิทธิ์การเปิดใช้งานระบบ (**root privilege requirement**): หากไม่รันโปรแกรมด้วยสิทธิ์ **root** (หรือคำสั่ง **sudo**) ระบบปฏิบัติการจะไม่ยินยอมให้ใช้การตั้งค่า **sched_fifo** และ **mlockall**ส่งผลให้ระบบตกไปใช้การจัดลำดับความสำคัญแบบปกติ (**sched_other**) ซึ่งทำให้เกิด **timing jitter** สูงขึ้นอย่างมาก

• ผลกระทบจาก **go async preemption**: ในโปรเซสภาษา **go** หากไม่ตั้งค่า **env godebug=asyncpreemptoff=1** ตอนสั่งรัน ตัว **go garbage collector /preemption** อาจเข้าแทรกการทำงานใน **cpu core 3** จนทำให้รอบเวลาของกล้องตก

4.2 แนวทางการแก้ไขปัญหาเบื้องต้น (troubleshooting)

• หากพบค่า jitter แกว่งผิดปกติ ให้ตรวจสอบว่ามีโปรเซสอื่นแทรกเข้ามาทำงานใน cpu core 3 หรือไม่ ด้วยคำสั่ง htop

• หากอ่านข้อมูลจาก shared memory ไม่ได้ ให้ตรวจสอบว่าโปรเซส writer และ camcap เปิดใช้งานเรียบร้อยแล้วหรือไม่

ส่วนที่ 5: แหล่งข้อมูล อุปกรณ์ และสิทธิ์การเข้าถึง (resources & access)

5.1 คลังโค้ดและไฟล์ตั้งค่า (code repository & config)

• ไฟล์โค้ดหลักและไฟล์ตั้งค่า configsensor.toml อยู่ในโฟลเดอร์โครงการบนเครื่อง raspberry pi 5 / cm5

5.2 รายการอุปกรณ์ฮาร์ดแวร์ที่ส่งมอบ (hardware assets)

• บอร์ด raspberry pi 5 / cm5 (ติดตั้งระบบปฏิบัติการ preempt_rt linux)

• เซ็นเซอร์ imu (icm-42688-p) เชื่อมต่อผ่าน spi

• เซ็นเซอร์ gps (u-blox m10) เชื่อมต่อผ่าน uart

• โมดูลกล้อง imx219 (csi camera)

5.3 สิทธิ์การเข้าถึงระบบ (access rights)

• ใช้สิทธิ์ root สำหรับรันโปรแกรมระบบเรียลไทม์

SHM Real-Time Pipeline

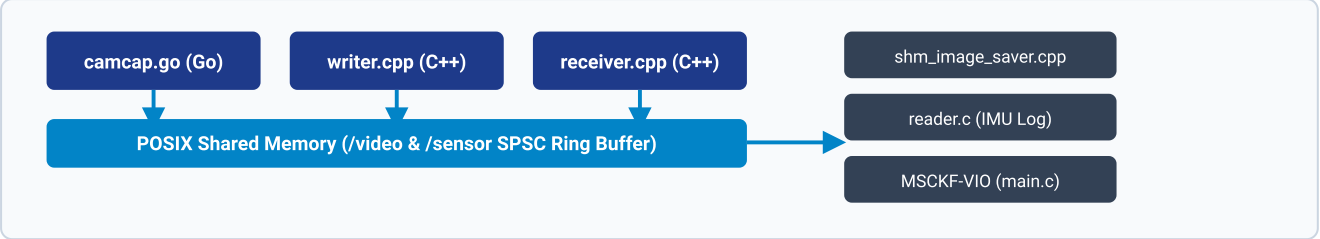
Sensor & Camera Data Acquisition Architecture for Raspberry Pi (PREEMPT_RT)

PREEMPT_RT / POSIX SHM

LOCK-FREE INTEROP	PRECISION	LANGUAGES	TARGET OS
SPSC Ring Buffer	**Nanosecond (ns)**	**C / C++ / Go**	**Linux RT Kernel**

1. System Architecture Overview

ระบบออกแบบขึ้นมาเพื่อรับข้อมูลความเร็วสูงจาก 3 แหล่ง ได้แก่ **Camera**, **IMU**, และ **GPS** บน Raspberry Pi ที่ใช้ PREEMPT_RT Kernel โดยส่งข้อมูลผ่าน **POSIX Shared Memory (SHM)** เพื่อวิเคราะห์ Latency และ Jitter โดยไม่ทำให้เกิด Non-deterministic Delays



2. Shared Memory Architecture & Memory Layout

ระบบแบ่ง Shared Memory เป็นสองส่วนแยกอิสระ โดยมี `static_assert` บังคับ Memory Layout และ Alignment ให้ตรงกันระหว่าง Go Struct และ C++ Alignment:

SHM Name	Size	Data Structure Layout	Producer	Consumer
`/sensor`	6 KB	`SensorData` = RingBuffer<IMU, 64> + RingBuffer<GPS, 64>	`writer.cpp` / `receiver.cpp`	`reader.c`, VIO App
`/video`	~8.1 MB	`VideoData` = RingBuffer<PNGImagePacket, 20>	`camcap.go`	`shm_image_saver.cpp`

3. Core Data Structures C/C++ Declarations

โครงสร้างข้อมูลหลักใน `shm_common.h` ที่มี Layout ตรงกับฝั่ง Go (Offset และ Padding ห้ามคลาดเคลื่อน):

```
// IMU Data Sample (writer.cpp -> /sensor)
struct IMUData {
    uint64_t timestamp;                     // [ns] CLOCK_MONOTONIC
    double w_RS_S_x, w_RS_S_y, w_RS_S_z;    // Gyroscope  [rad/s]
    double a_RS_S_x, a_RS_S_y, a_RS_S_z;    // Accelerometer [m/s²]
};

// Video Frame Packet (camcap.go -> /video)
struct PNGImagePacket {
    uint64_t timestamp;    // Offset 0
    uint32_t data_size;    // Offset 8
    uint8_t  data[409616]; // Offset 12 (Strictly no struct padding!)
};
```

4. Real-Time Determinism Rules

เพื่อป้องกัน Priority Inversion และ Unpredictable Latency ใน Real-Time Loops:

- **CPU Affinity:** ผูก RT Threads เข้ากับ CPU Cores ที่กำหนดเฉพาะด้วย `pthread_setaffinity_np`
- **SCHED_FIFO Scheduling:** กำหนด RT Priority สูงสุดแก่ Worker Threads
- **Memory Locking:** ใช้ `mlockall(MCL_CURRENT | MCL_FUTURE)` ร่วมกับ Stack Prefault เพื่อป้องกัน Page Fault
- **Lock-Free SPSC Ring:** อ่าน-เขียนข้อมูลโดยอาศัย Memory Barriers และ Atomic Indexing ไร้ Mutex Blocking
- **Spin-Wait Hybrid Sleep:** ใช้ `clock_nanosleep` นอนพักล่วงหน้า แล้ว Spin-wait ช่วง ~100µs สุดท้าย

5. Component Pipelines Summary

Camera Pipeline (camcap.go)
ดึง YUV420 Stream จาก `rpicam-vid` Decimate เฟรมลง 3 เท่า (60 FPS → 20 FPS) บีบอัด PNG ด้วย `libdeflate` ผ่าน cgo เพื่อประสิทธิภาพสูงสุด ปิด Go GC (`SetGCPercent(-1)`) เพื่อรักษาระดับ Latency

Component	Protocol / Source	Sampling Rate / Frequency	Key Features & Optimization
IMU Writer	SPI / I2C Bus	200 Hz (5ms period)	Auto-detect ICM-42688-P / MPU-6050, Low-latency DMA
GPS Receiver	UBX NAV-PVT Serial	10 Hz (Serial 115200)	Fletcher Checksum, SIMD `memmem` header search
Image Saver	SHM `/video`	Asynchronous Disk I/O	Background worker thread queue size = 300 frames

6. System Build & Execution Commands

คำสั่งและขั้นตอนในการคอมไพล์และเปิดรันระบบ Real-Time Pipeline:

```
# 1. Install Dependencies
sudo apt install g++ make libgpiodcxx-dev libgpiod-dev libdeflate-dev

# 2. Build All Components
make build

# 3. Execution (Runs all producers via taskset on dedicated CPU Cores)
make run

# 4. Stop All & Clean System Resources
make stop && make clean
```

7. Debugging & Analysis Tools

- `shm_probe.cpp` — เครื่องมือตรวจวัด Head, Tail, Used slots และ Timestamps ใน Ring Buffer
- `dump_layout.cpp` — ตรวจสอบ `sizeof` และ `offsetof` ของ Struct ต่างๆ เพื่อยืนยัน C++/Go Interop
- `difts.py` — สคริปต์ Python วิเคราะห์ Jitter และความต่างของ Timestamp ระหว่าง Sensor และ SHM

8. Log Files Output Reference

Log Path	Data Contents & Target Usage
`Log/process_timing.txt`	บันทึก 8 Timestamps ต่อ Frame (Deadline, Wait, Publish, Read, Encode)
`Log/process_duration_summary.txt`	สรุปสถิติ Processing Duration รวมของระบบ
`reader_imu.log`	ข้อมูล IMU Log ในรูปแบบมาตรฐาน EuRoC Format
`gps.log`	ข้อมูล GPS Decoded NAV-PVT Packets

9. Key Lessons Learned & Critical Notes

- **PNG Compression Limit:** ภาพ Grayscale 640x480 บีบอัดได้สูงสุด ~61-78KB ไม่สามารถย่อให้เล็กกว่า 50KB ได้ Frame Packet จึงต้องเผื่อไว้ ≥ 100KB
- **Go Struct Padding:** ห้ามใส่ `_ [4]byte` ใน Struct ของ Go เพราะจะทำให้ Offset ข้อมูลเคลื่อน และ C++ อ่าน Payload ผิดพลาด
- **Shared Memory Fail-safe:** หาก `shm_open` ล้มเหลว อาจเกิด Anonymous Private Mapping เงียบๆ ให้ตรวจสอบ Error Code เสมอ